accuracy. These functions are *double read()* and *double read(int times)*, respectively.

The code for File 1 (the header file *EFYLM35.h*) is:

```
#ifndef __EFYLM35__
#define __EFYLM35__
#include "Arduino.h"
class EFYLM35 {
     int _pin;
public:
     EFYLM35(int pin);
     ~EFYLM35();
     double read();
     double read(int times);
};
#endif
```

The file *EFYLM35.cpp* will define the variables and functions used in the library. In this example, we declared that one port pin is in input mode, so the user can choose the pin number to measure the analogue value from LM35. This function will read the analogue value using the inbuilt 12A DC on Arduino IDE. The output of an LM35 can be connected directly to an Arduino analogue input. Because the Arduino analogue-to-digital converter (ADC) has a resolution of 1024 bits, and the reference voltage is 5V, the equation used to calculate the temperature from the ADC value is:

```
Temp = [((5.0 * sensorValue)) / 1024) * 100.0.]
```

We can convert the read 12-bit digital value by multiplying with a constant of 0.48887585. The second function will ensure the accuracy of the temperature by taking the average of 'N' measurements.

The code for File 2 (the cpp file *EFYLM35.cpp*) is:

```
#include "EFYLM35.h"
EFYLM35:: EFYLM35(int pin): _pin(pin) {
     pinMode(pin, OUTPUT);
}

EFYLM35::~ EFYLM35(){ }
double EFYLM35::read() {
     int sensorValue = analogRead(_pin);
     double temperature = (sensorValue * 0.48875855);
     return temperature;
}
double EFYLM35::read(int times)
 {
     int sum = 0;
     for (int i = 0; i < times; i++) sum += analogRead(_pin);
     double average = (sum * 0.48875855) / times;
     return average;
}
```

```
  File 03: keywords.txt. This file is not necessary only for
users' information.
EFYLM35 KEYWORD1
read KEYWORD2
```

For simplicity in programming, we have not used any LCD interfacing. The temperature is viewed on the serial monitor of the IDE. Remember, you must choose the serial data transfer rate with 9600 bps, because our laptop or PC serial communication supported the RS 232C communication protocol. Here we have connected LM35 on the A0 channel and are taking the measurements five times.

```
#include <EFYLM35.h>
void setup()
{  Serial.begin(9600);
}
void loop()
{
  EFYLM35 sensor1(A0); // Sensor connected to pin A0
  double temperature = sensor1.read(); //Returns temperature
based on a single reading
  double avgtemperature = sensor1.read(5); //Returns average
temperature
  Serial.println(temperature);
  Serial.println(avgtemperature);
}
```

Final steps

1. Compile File 1 using C++ and save with extension *EFYLM35.h.*
2. Compile File 2 using C++ and save with extension *EFYLM35.cpp.*
3. Combine *EFYLM35.h*, *EFYLM35.cpp* and *keywords.txt* in a folder EFYLM35 and zip the folder using WinZip or WinRAR.
4. Open the Arduino IDE and go to *Sketch > Include library > Add.ZIP library > Choose our zip folder EFYLM35*. Now our library is added to the IDE.
5. Open a new file from the Arduino IDE and open *Sketch > Include library > Choose the EFYLM35 library from the list.* Now our header file will be included to that new sketch.
6. Write the program for interfacing the LM35 sensor using our own library (or use *EFY_LM35.ino* from the downloaded source code), and verify the output on the serial monitor.

The above steps and code help you to create and test your first library. You can try writing more complex libraries using the above process.

Note: The complete code can be downloaded from *https://opensourceforu.com/article_source_code/dec18/ard_lib.zip.*

END

By: K.M. Abubeker and Muth Sebastian

The authors are assistant professors in the department of ECE at the Amal Jyothi College of Engineering.